Docker Lab Practicals

Programs, Commands and Output Screenshots

DevOps Practical Session

Name: _____ Roll No.: _____

Contents

No.	Practical
1	Verify Docker Installation
2	Pull and List Docker Images
3	Create, Run, Stop and Remove a Container
4	Run an Ubuntu Container
5	Host a Website Using Nginx
6	Build a Custom Docker Image Using a Dockerfile
7	Demonstrate Docker Volumes
8	Demonstrate Docker Networking
9	Run a Container with Docker Compose

Practical 1: Verify Docker Installation

Aim: To verify that Docker is installed and running correctly.

Commands

```
docker --version
docker info
docker run hello-world
```

Explanation

- **docker --version** prints the installed Docker client version, and **docker version** shows both client and server details.
- **docker info** displays system-wide information: the active context, installed CLI plugins, storage driver, number of containers and images.
- **docker run hello-world** makes the Docker client contact the Docker daemon. The daemon pulls the *hello-world* image from Docker Hub (if it is not already local), creates a container from it, runs it, and streams the output back to the terminal.

Output Screenshots

```
C:\Users\Navanshu\OneDrive\Desktop\DockerPythonLab>docker --version
Docker version 29.8.2, build 7fc2dff

C:\Users\Navanshu\OneDrive\Desktop\DockerPythonLab>docker version
Client:
```

Figure 1.1 - docker --version

```
C:\Users\Navanshu\OneDrive\Desktop\DockerPythonLab>docker --version
Docker version 29.8.2, build 7fc2dff

C:\Users\Nityaam\OneDrive\Desktop\DockerPythonLab>docker version
Client:
 Version:           29.8.2
 API version:       1.56
 Go version:        go1.26.8
 Git commit:        7fc2dff
 Built:             Wed Sep 30 19:35:42 2026
 OS/Arch:           windows/amd64
 Context:           desktop-linux
```

Figure 1.2 - docker version - client details (Docker 29.8.2)

```
C:\Windows\System32\cmd.e

C:\Users\Navanshu\OneDrive\Desktop\DockerPythonLab>docker info
Client:
 Version:    29.8.2
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  agent: Docker AI Agent Runner (Docker Inc.)
    Version:  v1.144.0
    Path:     C:\Users\Navanshu\.docker\cli-plugins\docker-agent.exe
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.31.0
    Path:     C:\Users\Navanshu\.docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.37.2
    Path:     C:\Users\Navanshu\.docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.5.1
    Path:     C:\Users\Navanshu  \.docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container (Docker Inc.)
    Version:  0.0.47
    Path:     C:\Users\Navanshu  \.docker\cli-plugins\docker-debug.exe
  desktop: Docker Desktop commands (Docker Inc.)
    Version:  v0.4.4
    Path:     C:\Users\Navanshu\.docker\cli-plugins\docker-desktop.exe
  dhi: CLI for managing Docker Hardened Images (Docker Inc.)
    Version:  v0.0.7
    Path:     C:\Users\Navanshu\.docker\cli-plugins\docker-dhi.exe
  extension: Manages Docker extensions (Docker Inc.)
    Version:  v0.2.31
    Path:     C:\Users\Navanshu\.docker\cli-plugins\docker-extension.exe
  init: Creates Docker-related starter files for your project (Docker Inc.)
    Version:  v1.4.0
    Path:     C:\Users\Navanshu\.docker\cli-plugins\docker-init.exe
  mcp: Docker MCP Plugin (Docker Inc.)
    Version:  v0.44.1
    Path:     C:\Users\Navanshu\.docker\cli-plugins\docker-mcp.exe
  model: Docker Model Runner (Docker Inc.)
    Version:  v1.2.6
    Path:     C:\Users\Navanshu\.docker\cli-plugins\docker-model.exe
  offload: Docker Offload (Docker Inc.)
    Version:  v0.6.53
    Path:     C:\Users\Navanshu\.docker\cli-plugins\docker-offload.exe
  pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
    Version:  v0.2.4
    Path:     C:\Users\Navanshu  \.docker\cli-plugins\docker-pass.exe
  sandbox: "docker sandbox" is deprecated, use Docker Sandboxes instead (Docker Inc.)
    Version:  v0.13.0
    Path:     C:\Users\Navanshu  \.docker\cli-plugins\docker-sandbox.exe
  scout: Docker Scout (Docker Inc.)
```

Figure 1.3 - docker info - client, context and installed plugins

```
C:\Users\Navanshu\OneDrive\Desktop\DockerPythonLab>docker hello-world
docker: unknown command: docker hello-world

Run 'docker --help' for more information

C:\Users\Navanshu\OneDrive\Desktop\DockerPythonLab>docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Figure 1.4 - docker run hello-world - "Hello from Docker!" confirms the installation works

What We Learn

- Docker is installed correctly and the Docker daemon is running.
- Docker uses a client-server architecture: the CLI (client) sends commands to the daemon.
- An **image** is a read-only template; a **container** is a running instance created from an image.
- *docker run* can pull an image, create a container and start it in a single command.

Practical 2: Pull and List Docker Images

Aim: To download an image from Docker Hub and inspect the locally available images.

Commands

```
docker pull nginx
docker images

# Student task
docker pull ubuntu
docker images
```

Explanation

- **docker pull nginx** downloads the Nginx image from Docker Hub. No tag was given, so the default tag *latest* is used. Each line such as "Pull complete" is one image layer being downloaded.
- **docker images** lists the images stored locally with their ID, size on disk and content size.
- Pulling nginx a second time reports "Image is up to date", because the layers already exist locally and are not downloaded again.
- Student task: nginx (243 MB on disk) is a ready-made web server image, while ubuntu (163 MB) is a base operating-system image. Both appear in the list and in Docker Desktop.

Output Screenshots

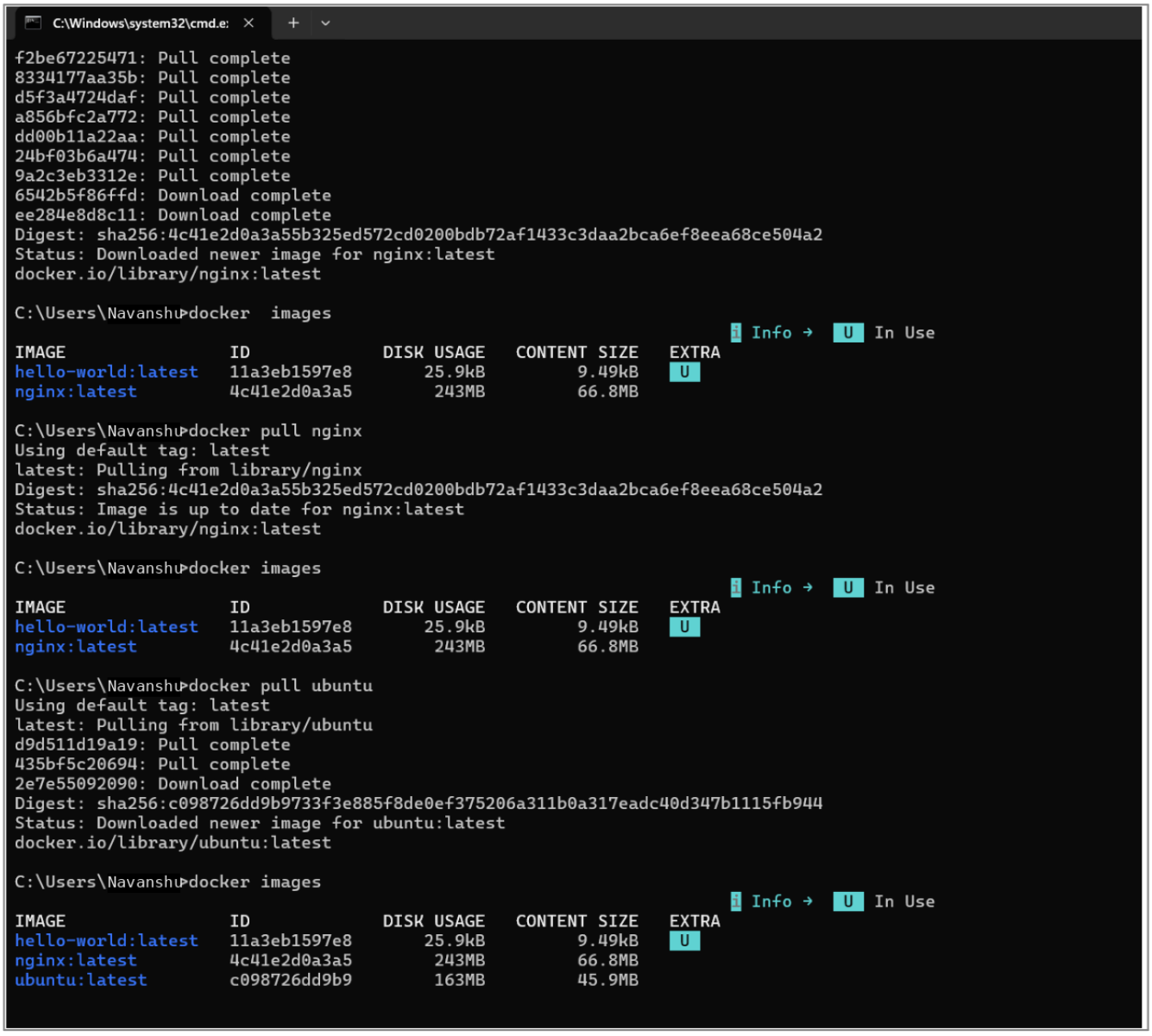


Figure 2.1 - Pulling nginx and ubuntu, and listing images after each pull

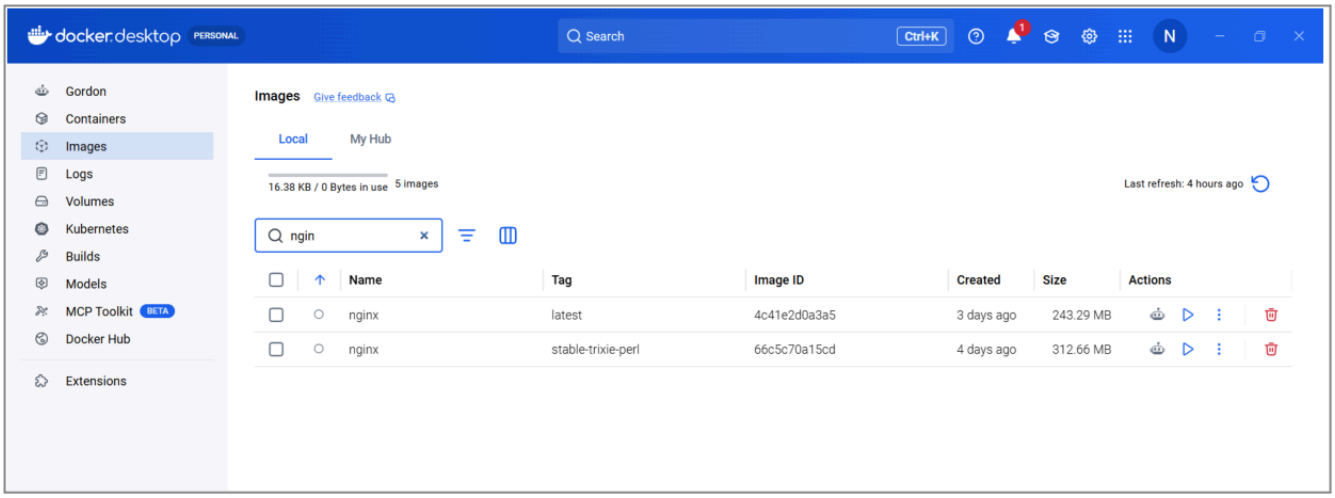


Figure 2.2 - Docker Desktop (Images tab) - nginx images shown locally

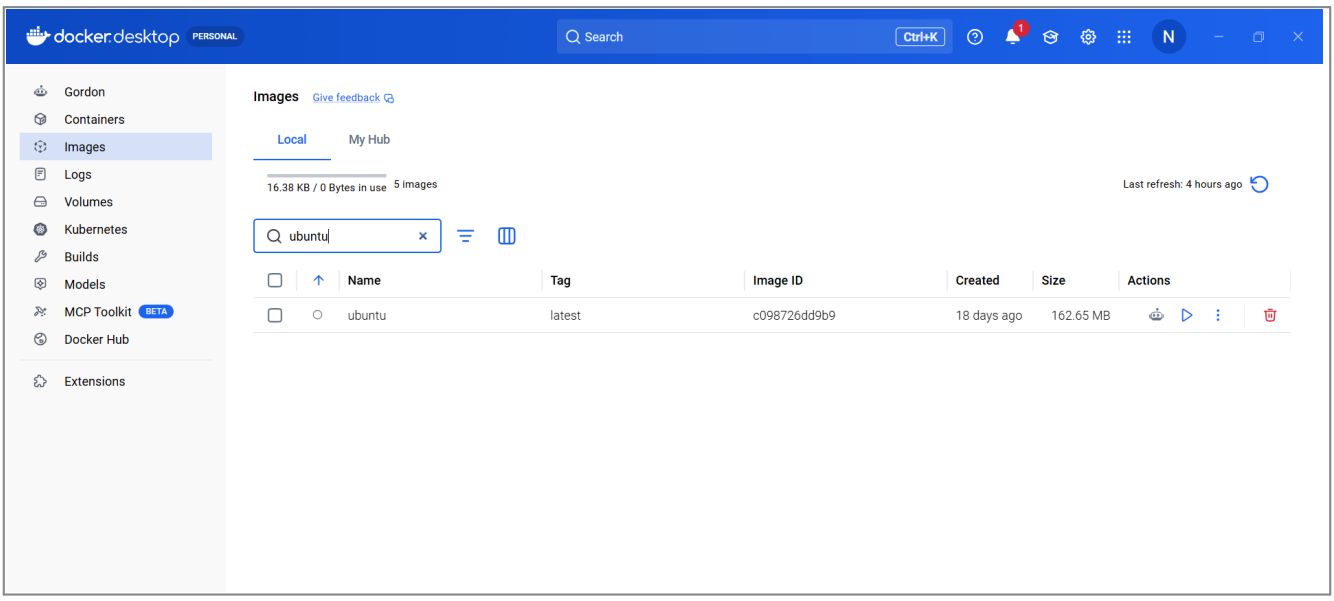


Figure 2.3 - Docker Desktop (Images tab) - ubuntu:latest shown locally

What We Learn

- Images are downloaded from a registry (Docker Hub) and stored locally.
- Images are built from layers, and layers that already exist are reused rather than downloaded again.
- Tags (such as *latest*) identify versions of an image.
- Different images serve different purposes: application images (nginx) and base OS images (ubuntu).

Practical 3: Create, Run, Stop and Remove a Container

Aim: To demonstrate the Docker container lifecycle.

Commands

```
docker run -d --name myweb nginx
docker ps
docker ps -a
docker stop myweb
docker start myweb
docker rm -f myweb
```

Explanation

- **docker run -d --name myweb nginx** creates and starts a container in the background (*-d* = detached) and names it *myweb*.
- **docker ps** lists only running containers; **docker ps -a** lists all containers, including stopped ones.
- **docker stop myweb** stops the container (it disappears from *docker ps*); **docker start myweb** starts the same container again.
- **docker rm -f myweb** force-removes the container, after which *docker ps -a* shows an empty list. (The mistyped *dockee* command in the screenshot simply failed and was retyped correctly.)

Output Screenshots

```
C:\Users\Navanshu>docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\Navanshu>docker run -d --name myweb nginx
a0692746fc6819dbb74924f0d1eaad733d5a31f63ae0d6a7a91617af1365520b

C:\Users\Navanshu>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS     NAMES
a0692746fc68   nginx     "/docker-entrypoint.…"   5 seconds ago   Up 5 seconds   80/tcp    myweb

C:\Users\Navanshu>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS     NAMES
a0692746fc68   nginx     "/docker-entrypoint.…"   9 seconds ago   Up 9 seconds   80/tcp    myweb

C:\Users\Navanshu>docker stop myweb
myweb

C:\Users\Navanshu>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\Navanshu>docker start myweb
myweb

C:\Users\Navanshu >docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS         PORTS     NAMES
a0692746fc68   nginx     "/docker-entrypoint.…"   42 seconds ago   Up 4 seconds   80/tcp    myweb

C:\Users\Navanshu>dockee rm -f myweb
'dockee' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Navanshu>docker rm -f myweb
myweb

C:\Users\Navanshu>docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\Navanshu>
```

Figure 3.1 - Container lifecycle: running, stopped, restarted and removed

What We Learn

- A container has a lifecycle: created, running, stopped and removed.
- Stopping a container does not delete it; it can be started again until it is removed.
- *docker ps* versus *docker ps -a* shows the difference between running and all containers.

Practical 4: Run an Ubuntu Container

Aim: To execute Linux commands inside a Docker container on Windows.

Commands

```
docker run -it --name ubuntu-lab ubuntu bash

# Inside the Ubuntu terminal:
cat /etc/os-release
pwd
ls
mkdir devops
cd devops
touch lab.txt
ls
exit

# Remove the stopped container:
docker rm ubuntu-lab
```

Explanation

- **docker run -it --name ubuntu-lab ubuntu bash** starts an Ubuntu container with an interactive terminal (*-i* keeps input open, *-t* allocates a terminal) and runs *bash* inside it.
- Inside the container we ran normal Linux commands: *cat /etc/os-release* shows the OS (Ubuntu 26.04 LTS), *pwd* and *ls* explore the file system, and *mkdir*, *cd* and *touch* create a folder and a file.
- **exit** leaves the shell, which stops the container. **docker rm ubuntu-lab** then deletes the stopped container.

Output Screenshots

```
C:\Users\Navanshu>docker run -it --name ubuntu-lab ubuntu bash
root@a8af8f6085e0:/# pwd
/
root@a8af8f6085e0:/# ls
bin   dev  home  lib64  mnt  proc  run   srv  tmp  var
boot  etc  lib   media  opt  root  sbin  sys  usr
root@a8af8f6085e0:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04.1 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
root@a8af8f6085e0:/# mkdir devops
root@a8af8f6085e0:/# cd devops
root@a8af8f6085e0:/devops# ls
root@a8af8f6085e0:/devops# touch lab.txt
root@a8af8f6085e0:/devops# ls
lab.txt
root@a8af8f6085e0:/devops# pwd
/devops
root@a8af8f6085e0:/devops# exit
exit

C:\Users\Navanshu>docker rm ubuntu-lab
ubuntu-lab

C:\Users\Navanshu>
```

Figure 4.1 - Interactive Ubuntu container: commands run inside, then container removed

What We Learn

- A full Linux environment can run inside a container without installing a separate operating system or virtual machine.
- Containers share the host's kernel, which is why they start quickly and are lightweight.
- Files created inside a container's own file system are lost when the container is removed, which is why volumes are needed (Practical 7).

Practical 5: Host a Website Using Nginx

Aim: To deploy a web server using Docker.

Commands

```
# Step 1: Start the web server
docker run -d --name mywebsite -p 8080:80 nginx

# Step 2: Open http://localhost:8080 in the browser

# Step 3: Inspect the container
docker ps
docker logs mywebsite
docker inspect mywebsite

# Step 4: Stop and remove it
docker rm -f mywebsite
```

Explanation

- **docker run -d --name mywebsite -p 8080:80 nginx** starts Nginx in the background. *-p 8080:80* maps port 8080 on the host to port 80 inside the container, so the site is reachable at http://localhost:8080.
- The browser shows the "Welcome to nginx!" page, proving the web server is running inside the container.
- **docker ps** shows the running container and its port mapping. **docker logs** shows Nginx start-up messages and the access log (the browser's *GET /* request returned 200). **docker inspect** prints the full configuration as JSON: state, network mode, IP address and port bindings.
- **docker rm -f** stops and removes the container.

Output Screenshots

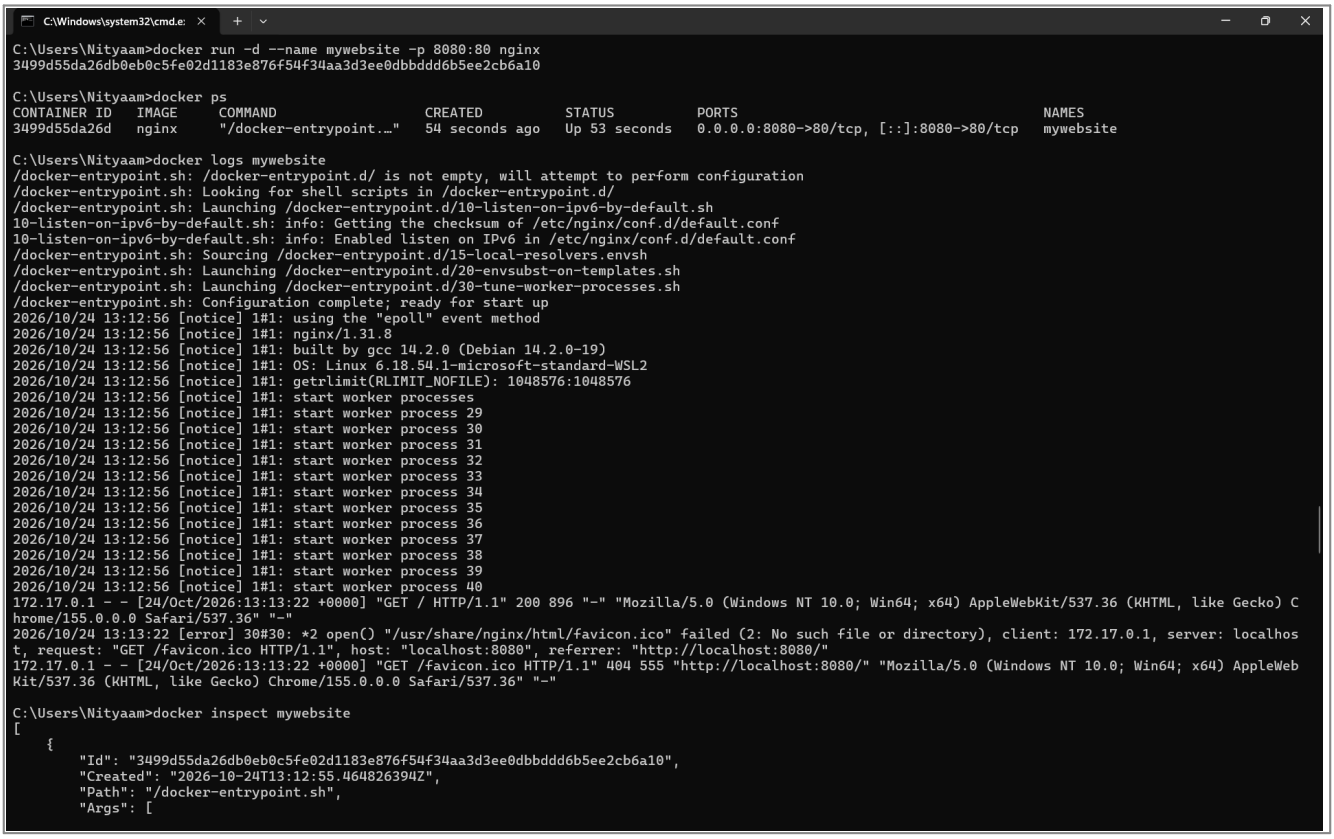


Figure 5.1 - Step 1 and 3 - container started on port 8080; docker ps, docker logs and docker inspect

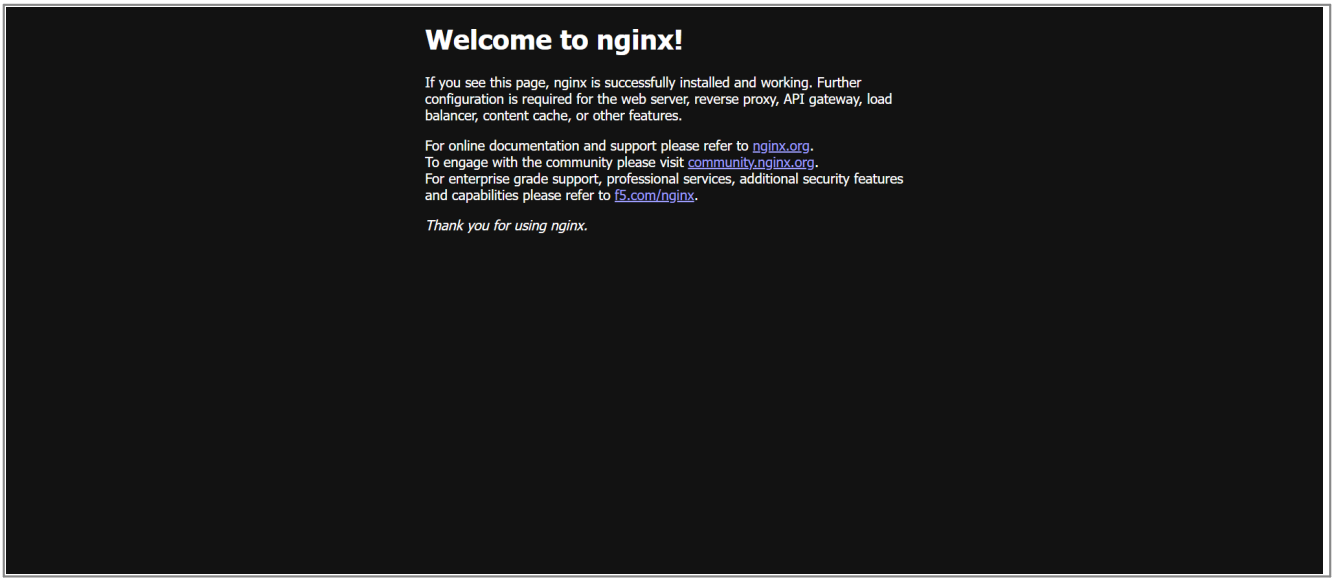


Figure 5.2 - Step 2 - Nginx welcome page served from the container

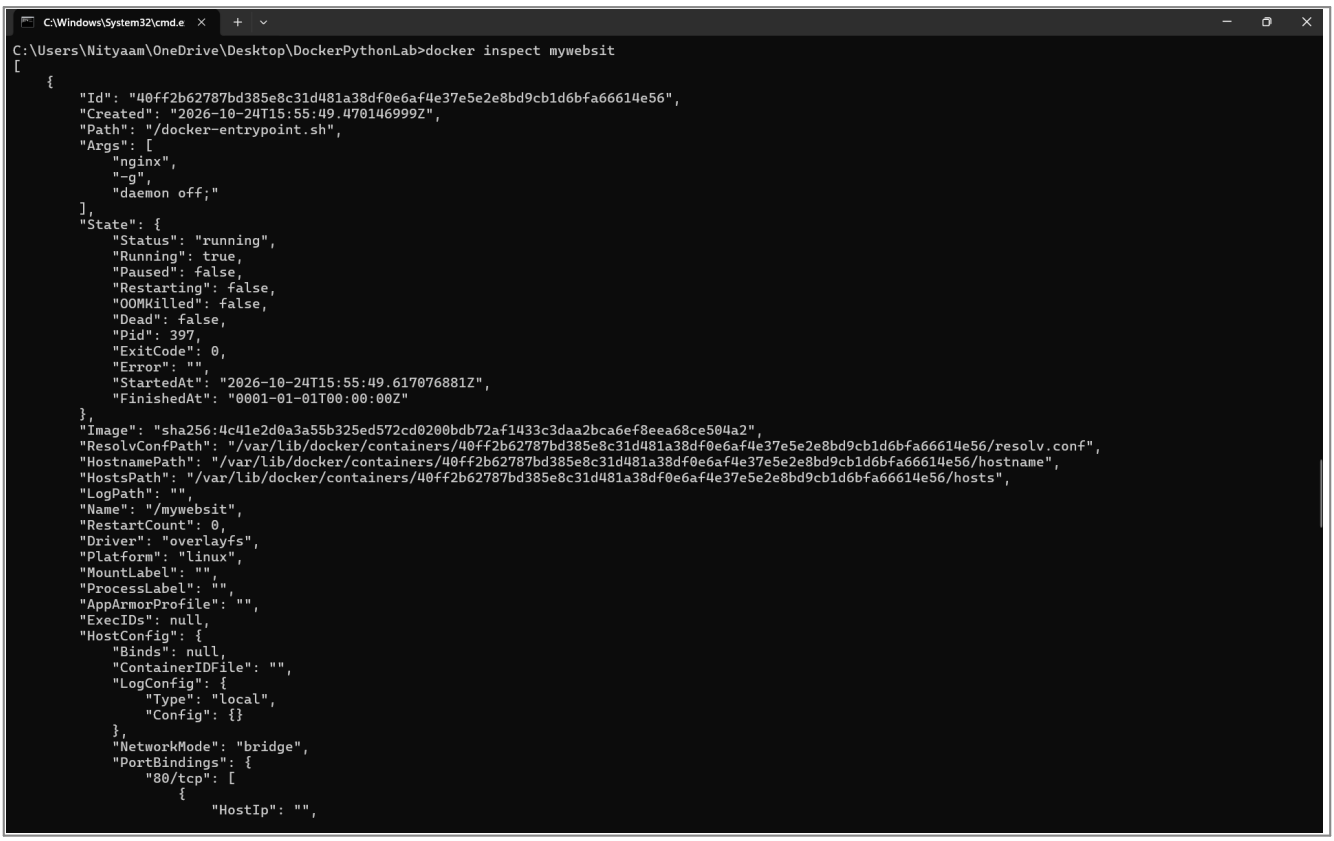


Figure 5.3 - Step 3 - docker inspect output (state running, bridge network, port bindings)



Figure 5.4 - Step 4 - container force-removed with docker rm -f

What We Learn

- A web server can be deployed with one command, without installing or configuring Nginx manually on Windows.
- Port mapping (*-p host:container*) exposes a service running in a container to the host machine.
- *docker logs* and *docker inspect* are the main tools for monitoring and debugging a container.

Practical 6: Build a Custom Docker Image Using a Dockerfile

Aim: To create a custom image for a simple Python program.

Commands

```
# In the DockerPythonLab folder, create app.py and Dockerfile, then run:
docker build -t python-lab .
docker images
docker run --rm python-lab
```

Explanation

- A **Dockerfile** is a text recipe for building an image. Here it uses *FROM python:3.12-slim* (base image), *WORKDIR /app* (working folder), *COPY app.py .* (copy the program in) and *CMD* (the command to run when a container starts).
- **docker build -t python-lab .** reads the Dockerfile in the current folder and builds an image tagged *python-lab*. The build output shows each instruction executed as a step.
- **docker images** now lists *python-lab:latest* alongside the other images.
- **docker run --rm python-lab** runs the program in a new container, prints its output, and automatically removes the container when it finishes (*--rm*).

Output Screenshots

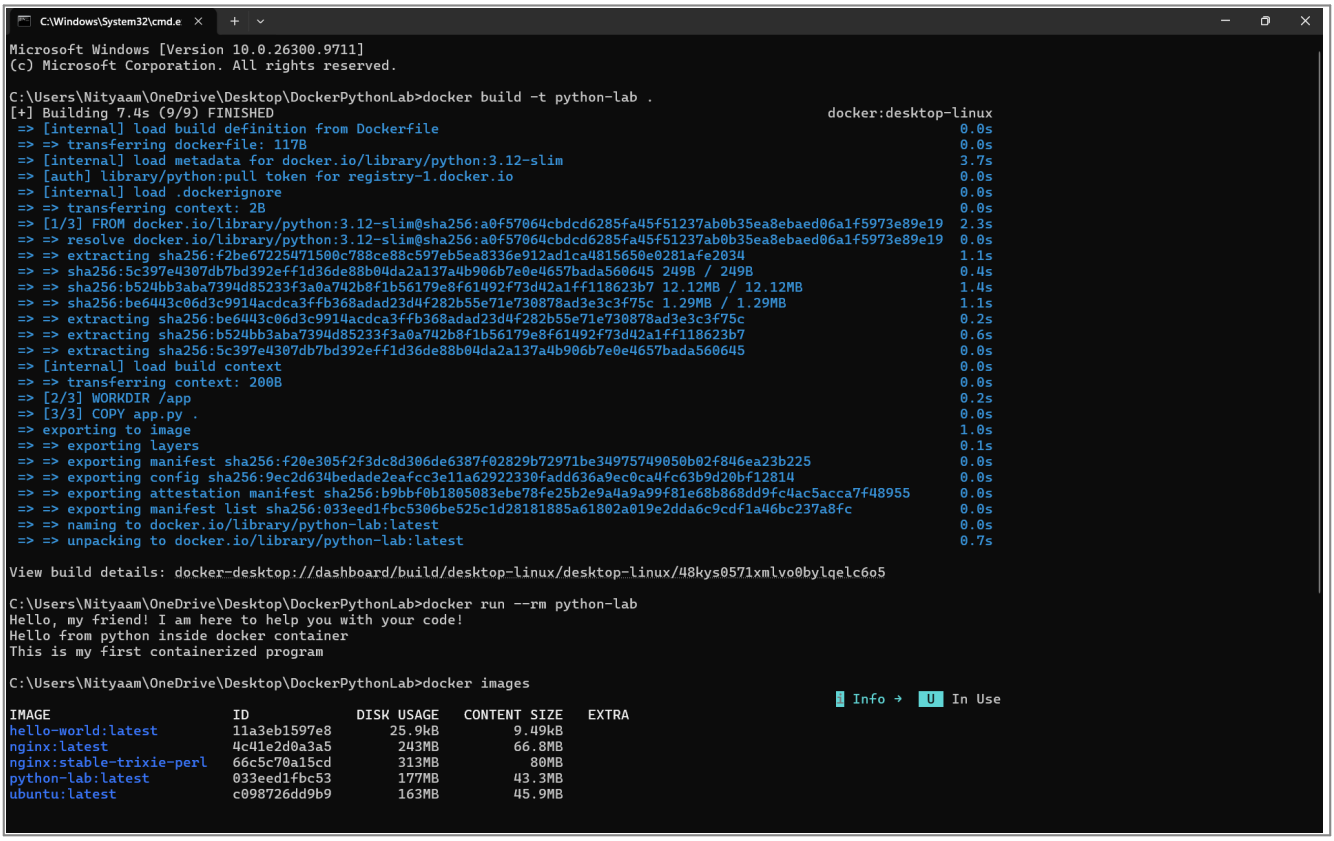


Figure 6.1 - Image built from the Dockerfile, container run, and python-lab listed in docker images

What We Learn

- How to package an application into a custom image using a Dockerfile.
- The meaning of the main Dockerfile instructions: FROM, WORKDIR, COPY and CMD.
- Image building is done in layers, and an image is named with a tag.
- The same image runs identically on any machine with Docker, which makes applications portable.

Practical 7: Demonstrate Docker Volumes

Aim: To understand persistent data storage.

Commands

```
docker volume create studentdata
docker run --rm -v studentdata:/data ubuntu bash -c "echo Docker Lab Data > /data/test.txt"
docker run --rm -v studentdata:/data ubuntu cat /data/test.txt
docker volume ls
```

Explanation

- **docker volume create studentdata** creates a named volume, a storage area managed by Docker outside any container.
- The first *docker run --rm -v studentdata:/data ...* mounts the volume at */data* and writes *test.txt* into it. Because of *--rm*, this container is deleted as soon as it finishes.
- A second, brand-new container mounts the same volume and reads the file with *cat*, printing "Docker Lab Data". (The first read attempt used the wrong name *test.text* and failed; the corrected command worked.)
- **docker volume ls** confirms that the volume still exists.

Output Screenshots

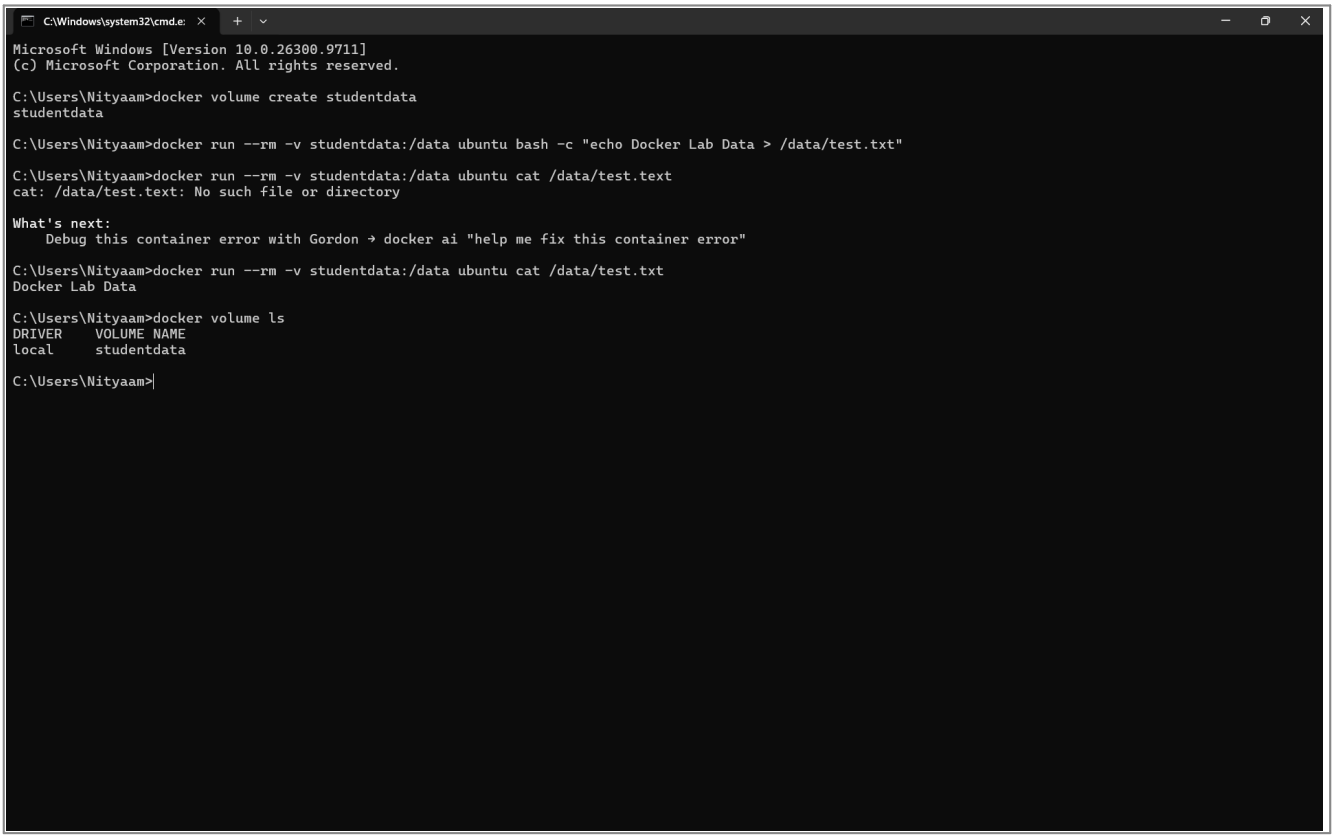


Figure 7.1 - Data written to the named volume survives after the first container is removed (a typo in test.text gave an error, then test.txt printed "Docker Lab Data")

What We Learn

- Containers are temporary: data inside them disappears when they are removed.
- A Docker volume stores data outside the container so it persists beyond the container's lifecycle.
- The same volume can be mounted into different containers to share data.

Practical 8: Demonstrate Docker Networking

Aim: To create a custom network for containers.

Commands

```
docker network create devops-network
docker run -d --name web1 --network devops-network nginx
docker network ls
docker network inspect devops-network

# Clean up:
docker rm -f web1
docker network rm devops-network
```

Explanation

- **docker network create devops-network** creates a user-defined bridge network.
- **docker run -d --name web1 --network devops-network nginx** starts a container attached to that network (named webOfKingNavanshu in the screenshot).
- **docker network ls** lists the default networks (bridge, host, none) and the new *devops-network*.
- **docker network inspect** shows the subnet (172.18.0.0/16), the gateway (172.18.0.1) and the connected container with its IP address (172.18.0.2).
- Clean-up: the container is removed first with *docker rm -f*, then the network with *docker network rm*. A network cannot be removed while containers are still attached.

Output Screenshots

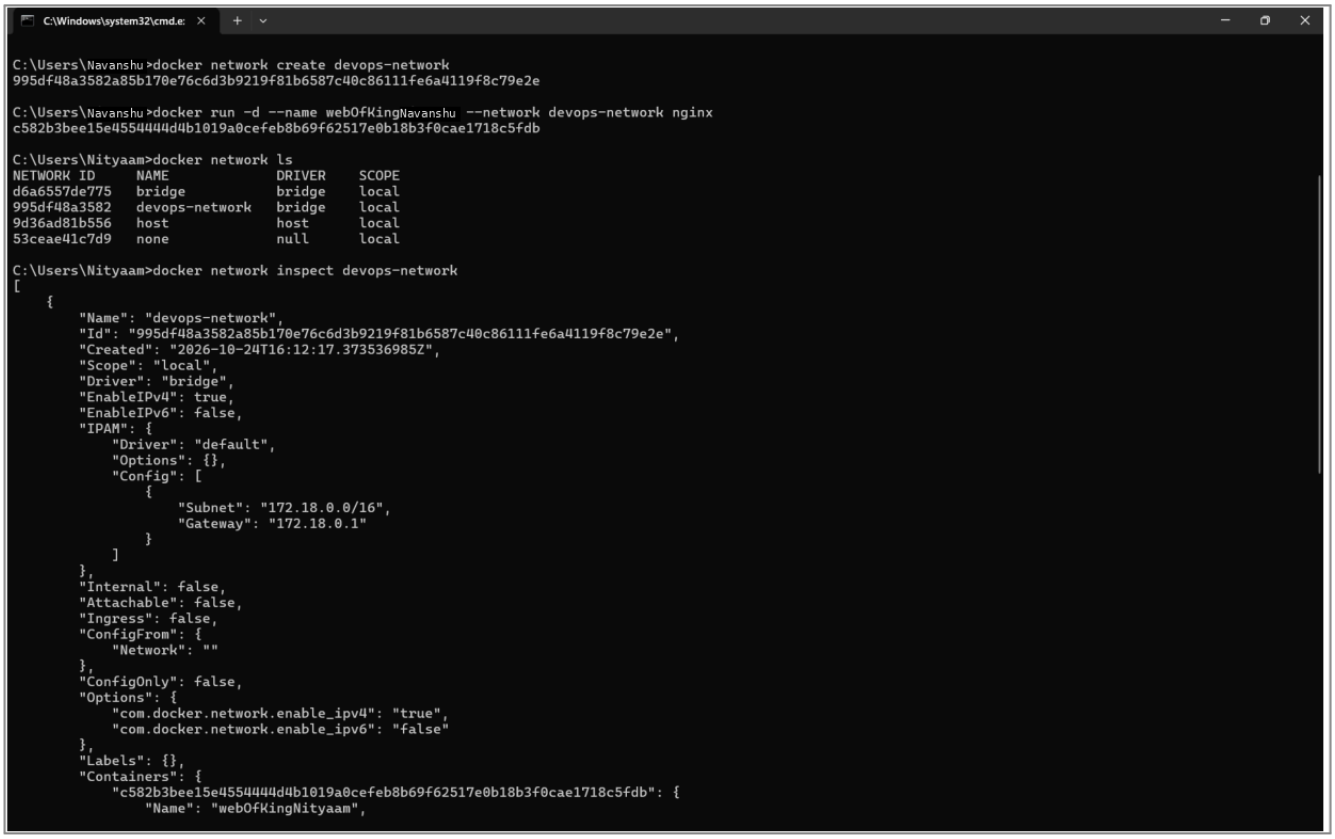


Figure 8.1 - Network created, container attached, docker network ls and docker network inspect

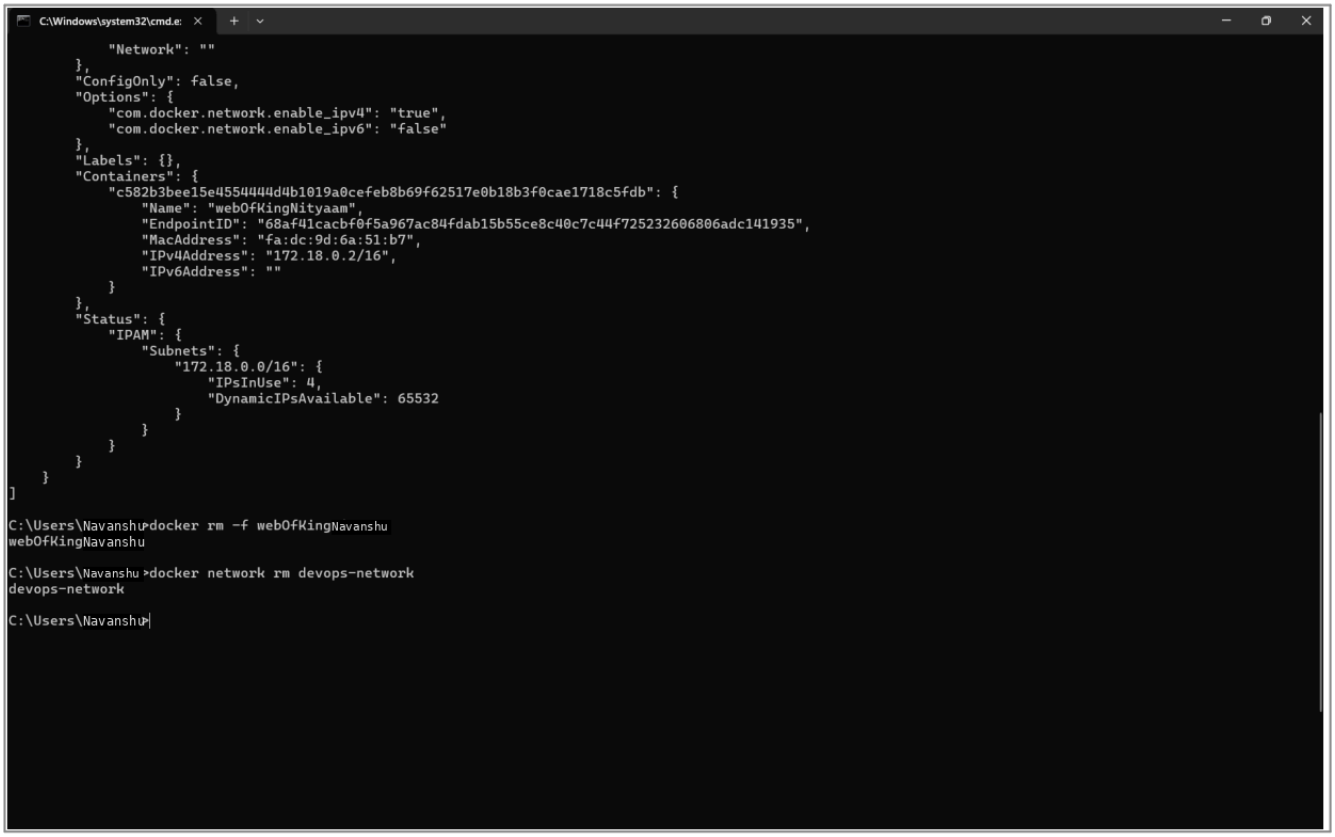


Figure 8.2 - End of inspect output (container IP 172.18.0.2) and clean-up of container and network

What We Learn

- Docker containers can be attached to a user-defined network.
- Each network has its own subnet and gateway, and each container gets its own IP address.
- Containers on the same user-defined bridge network can talk to each other using container names.
- Resources must be cleaned up in the right order: containers first, then networks.

Practical 9: Run a Container with Docker Compose

Aim: To launch an application service using a Compose configuration.

Commands

```
# Create compose.yaml in the DockerPythonLab folder, then run:
docker compose up -d
docker compose ps

# Open http://localhost:8080 to view the website

docker compose down
```

Explanation

- **compose.yaml** describes the application declaratively: one service called *web* that uses the *nginx* image and maps port 8080 to 80.
- **docker compose up -d** reads the file and creates everything it needs: a default network (*dockerpythonlab_default*) and the container (*dockerpythonlab-web-1*), then starts it in the background.
- **docker compose ps** shows the service status and port mapping. The Nginx welcome page opens at http://localhost:8080.
- **docker compose down** stops and removes the container and the network that Compose created.

Output Screenshots

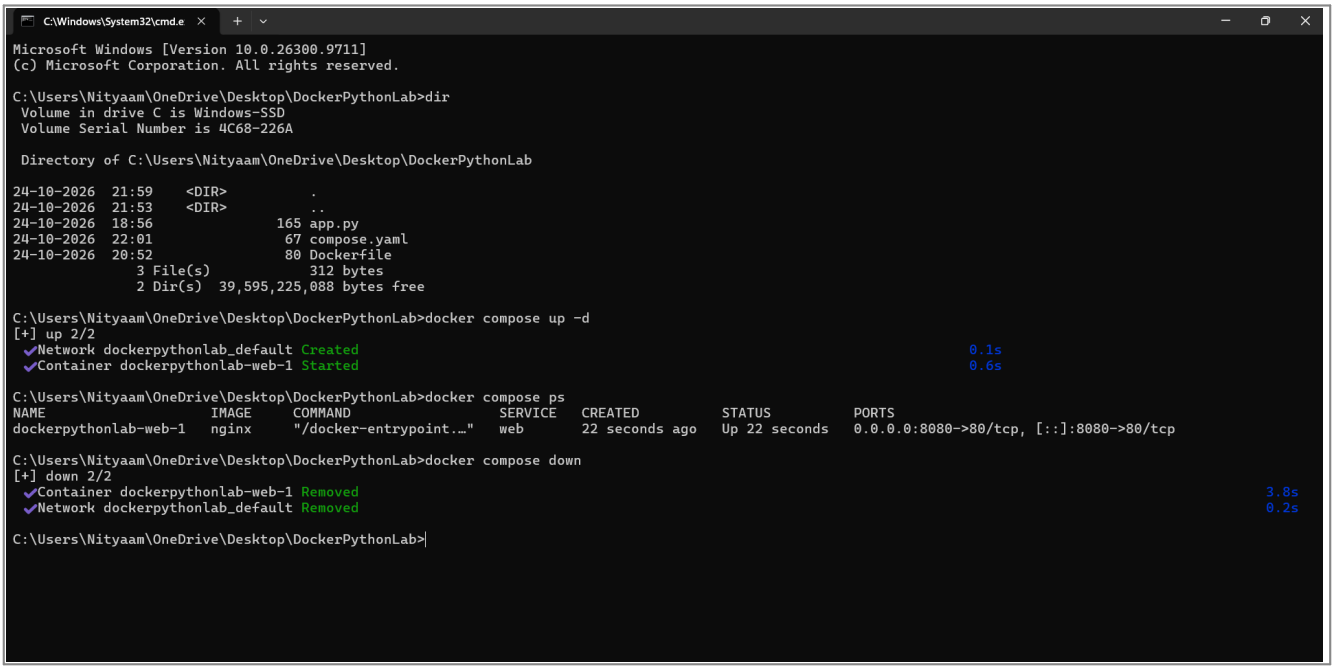


Figure 9.1 - docker compose up -d, docker compose ps and docker compose down

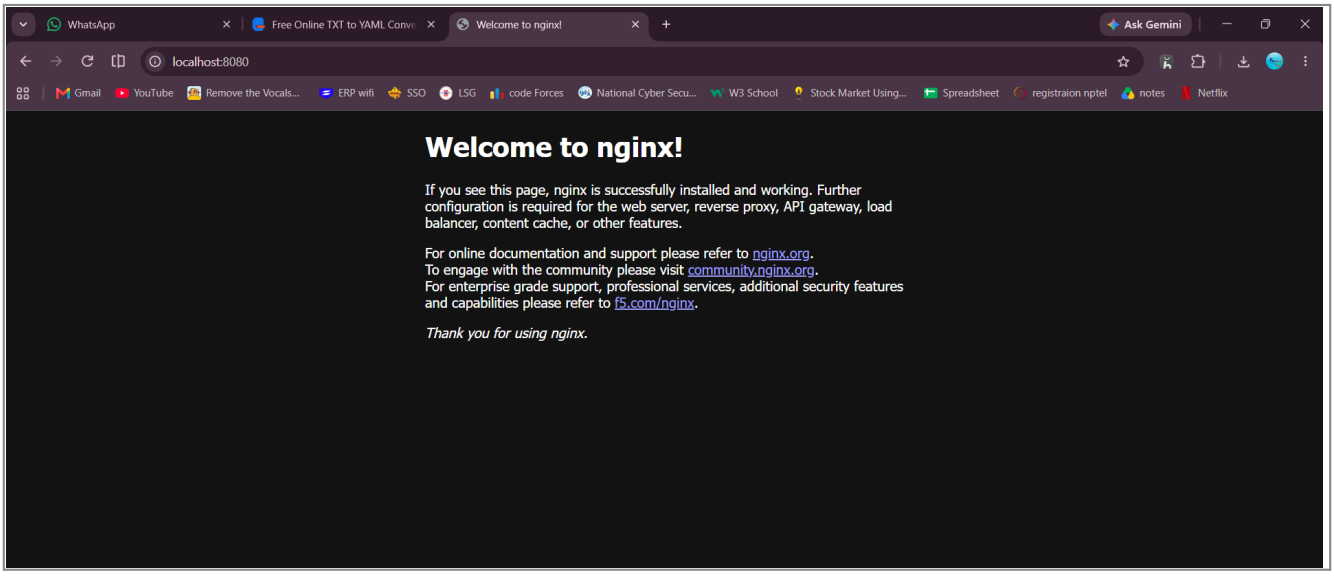


Figure 9.2 - Nginx welcome page at localhost:8080 served by the Compose service

What We Learn

- Docker Compose defines and manages application services declaratively in a YAML file.
- One command (*up*) starts everything and one command (*down*) removes it, so setups are repeatable.
- Compose is the foundation for multi-container applications such as a full stack app with a web server, back end and database.